

QuickSort

Aula 6

Fábio Henrique Viduani Martinez

Faculdade de Computação
Universidade Federal de Mato Grosso do Sul

Projeto e Análise de Algoritmos I

Conteúdo da aula

- 1 Motivação
- 2 QuickSort
- 3 Desempenho do QUICKSORT
- 4 Versão aleatorizada do QUICKSORT
- 5 Análise do QUICKSORT
- 6 Exercícios

Introdução

- ▶ o QUICKSORT também é chamado de **algoritmo de ordenação por separação**
- ▶ o QUICKSORT tem tempo de pior caso $\Theta(n^2)$ sobre um vetor de entrada com n elementos
- ▶ é o algoritmo de ordenação mais usado na prática devido a sua eficiência na média: seu tempo de execução *esperado* é $\Theta(n \lg n)$ e os fatores constantes escondidos na notação assintótica são muito pequenos
- ▶ usa espaço auxiliar constante para executar a ordenação

Divisão-e-conquista

Divida: rearranje o vetor $A[p..r]$ em dois subvetores $A[p..q-1]$ e $A[q+1..r]$ tal que cada elemento de $A[p..q-1]$ é menor ou igual a $A[q]$, que é, por sua vez, menor ou igual a cada elemento de $A[q+1..r]$. Compute o índice q como parte deste processo de partição.

Conquiste: ordene os dois subvetores $A[p..q-1]$ e $A[q+1..r]$ por chamadas recursivas ao QUICKSORT.

Combine: “cole” os subvetores $A[p..q-1]$, $A[q]$ e $A[q+1..r]$ e obtenha imediatamente o vetor $A[p..r]$ ordenado

Problema da separação

- ▶ a chave do algoritmo é a operação de separação

Problema da separação

Dado um vetor de números inteiros $A[p..r]$, rearranje esse vetor em dois subvetores $A[p..q-1]$ e $A[q+1..r]$ de tal forma que cada elemento de $A[p..q-1]$ é menor ou igual a $A[q]$, que é, por sua vez, menor ou igual a cada elemento de $A[q+1..r]$. Compute o índice q como parte deste processo.

Algoritmo PARTITION

```
PARTITION( $A, p, r$ )  
1.  $x \leftarrow A[r]$   
2.  $i \leftarrow p - 1$   
3. para  $j \leftarrow p$  até  $r - 1$   
4.   se  $A[j] \leq x$   
5.      $i \leftarrow i + 1$   
6.     troque  $A[i]$  com  $A[j]$   
7. troque  $A[i + 1]$  com  $A[r]$   
8. devolva  $i + 1$ 
```

Ilustração da execução do algoritmo PARTITION

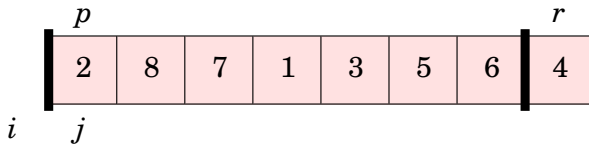


Ilustração da execução do algoritmo PARTITION

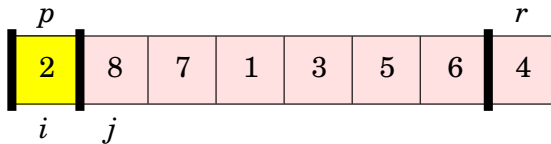


Ilustração da execução do algoritmo PARTITION

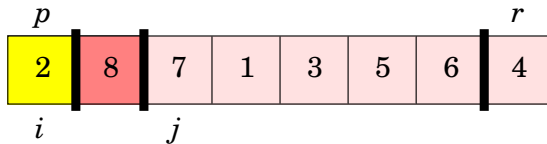


Ilustração da execução do algoritmo PARTITION

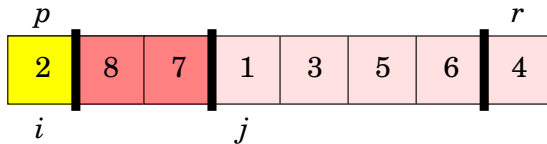


Ilustração da execução do algoritmo PARTITION

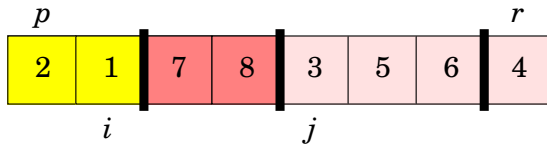


Ilustração da execução do algoritmo PARTITION

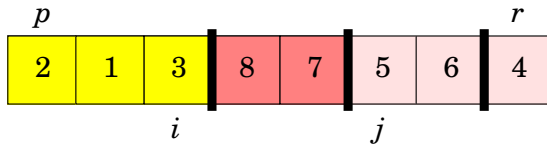


Ilustração da execução do algoritmo PARTITION

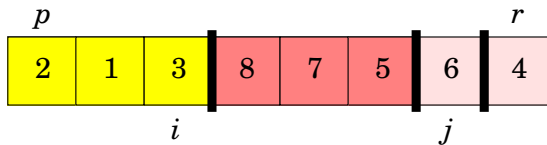


Ilustração da execução do algoritmo PARTITION

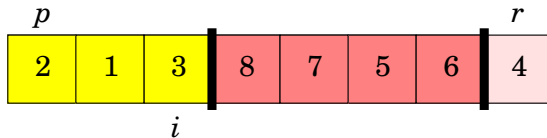
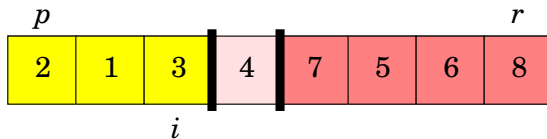


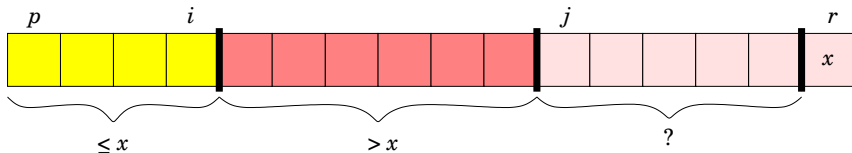
Ilustração da execução do algoritmo PARTITION



Invariante do algoritmo PARTITION

- ▶ o algoritmo PARTITION sempre seleciona um elemento $x = A[r]$ como um **pivô** em torno do qual a separação do vetor $A[p..r]$ será realizada
- ▶ durante a execução do algoritmo PARTITION, o vetor $A[p..r]$ é particionado em 4 regiões
- ▶ no começo de cada iteração da estrutura de repetição das linhas 3–6, as regiões satisfazem certas propriedades, que podem ser descritas como o invariante da estrutura de repetição

Invariante do algoritmo PARTITION



Invariante do algoritmo PARTITION

No começo de cada iteração da estrutura de repetição das linhas 3–6, para qualquer índice k do vetor A ,

1. Se $p \leq k \leq i$, então $A[k] \leq x$.
 2. Se $i + 1 \leq k \leq j - 1$, então $A[k] > x$.
 3. Se $k = r$, então $A[k] = x$.
- ▶ os índices entre j e $r - 1$ não são cobertos por qualquer dos 3 casos e os valores nesses compartimentos não têm relação direta com o pivô x
 - ▶ precisamos mostrar que este invariante é verdadeiro antes da primeira iteração, que cada iteração da estrutura de repetição mantém o invariante e que o invariante fornece uma propriedade útil para mostrar a correção do algoritmo quando a estrutura de repetição termina

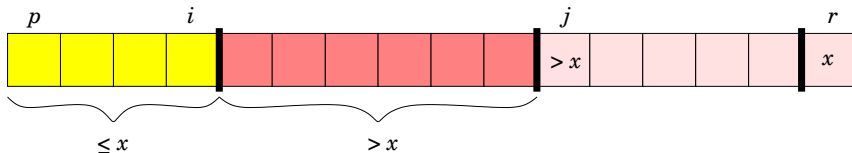
Correção do algoritmo PARTITION

Inicialização: Antes da primeira iteração da estrutura de repetição, $i = p - 1$ e $j = p$. Como não há valores armazenados no vetor A entre p e i e entre $i + 1$ e $j - 1$, as primeiras duas condições do invariante são trivialmente satisfeitas. A atribuição na linha 1 satisfaz a terceira condição.

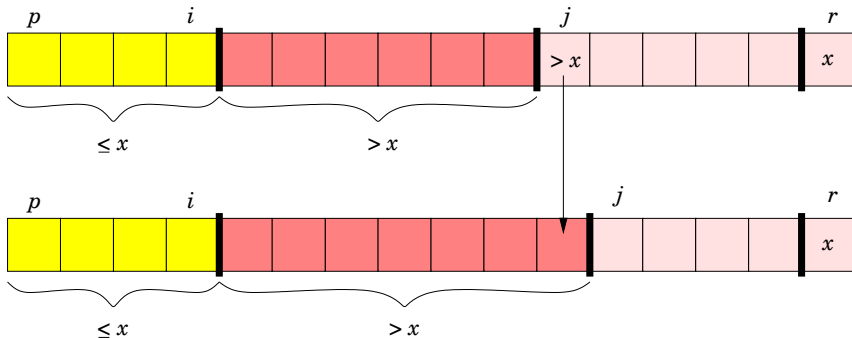
Correção do algoritmo PARTITION

Manutenção: Temos de considerar dois casos que dependem do resultado do teste da linha 4. Se $A[j] > x$ então j é incrementado. Depois que j é incrementado, a condição 2 é satisfeita para $A[j-1]$ e todos os outros elementos permanecem não modificados. (...)

Correção do algoritmo PARTITION



Correção do algoritmo PARTITION



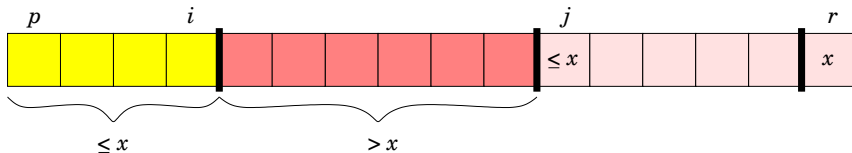
Correção do algoritmo PARTITION

Manutenção: Temos de considerar dois casos que dependem do resultado do teste da linha 4. Se $A[j] > x$ então j é incrementado. Depois que j é incrementado, a condição 2 é satisfeita para $A[j-1]$ e todos os outros elementos permanecem não modificados.

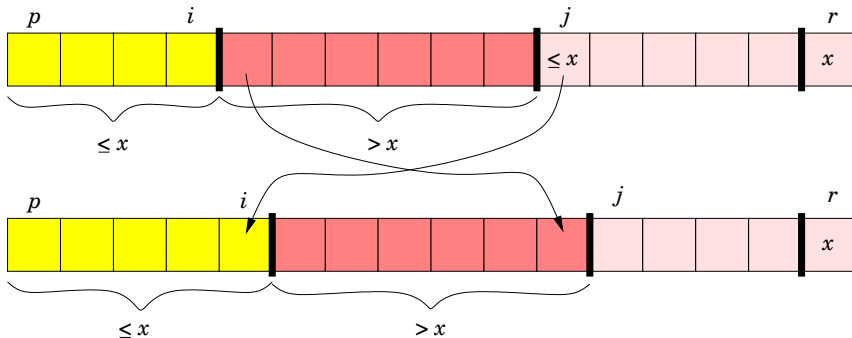
Se $A[j] \leq x$ então i é incrementado, $A[i]$ é trocado com $A[j]$ e depois j é incrementado. Devido à troca, temos agora que $A[i] \leq x$ e a condição 1 é satisfeita.

Similarmente, temos também que $A[j-1] > x$, já que o elemento que foi trocado em $A[j-1]$ é, pelo invariante, maior que x .

Correção do algoritmo PARTITION



Correção do algoritmo PARTITION



Correção do algoritmo PARTITION

Término: No final, $j = r$. Portanto, toda entrada no vetor está em um dos 3 conjuntos descritos pelo invariante e temos separado os valores do vetor em 3 conjuntos: aqueles menores ou iguais a x , aqueles maiores que x e um conjunto unitário contendo x .

Correção do algoritmo PARTITION

- ▶ as duas últimas linhas do algoritmo PARTITION trocam o pivô com o elemento mais à esquerda do conjunto dos elementos maiores que x , movendo portanto o pivô para a posição correta no vetor particionado e devolvendo o novo índice do pivô
- ▶ a saída do algoritmo PARTITION satisfaz, então, as especificações da etapa da divisão

Tempo de execução do algoritmo PARTITION

- ▶ como cada elemento do subvetor $A[p..r-1]$ é comparado uma única vez, na linha 4, com o pivô $A[r] = x$, o tempo de execução do algoritmo PARTITION é $\Theta(n)$, onde $n = r - p + 1$

Algoritmo QUICKSORT

QUICKSORT(A, p, r)

1. **se** $p < r$
2. $q \leftarrow \text{PARTITION}(A, p, r)$
3. QUICKSORT($A, p, q - 1$)
4. QUICKSORT($A, q + 1, r$)

Tempo de execução do algoritmo QUICKSORT

- ▶ o desempenho do QUICKSORT depende se a separação é balanceada ou não, que por sua vez depende de quais elementos são usados para realizar a separação
- ▶ se a separação é balanceada, o algoritmo é assintoticamente tão rápido quanto o MERGESORT
- ▶ se a separação não é balanceada, o algoritmo pode ser assintoticamente tão lento quanto o INSERTIONSORT

Separação de pior caso

- ▶ o comportamento de pior caso do QUICKSORT ocorre quando a separação produz um subproblema com $n - 1$ elementos e um outro com 0 elementos
- ▶ considere que esta separação desbalanceada ocorra em cada chamada recursiva
- ▶ o custo da separação é $\Theta(n)$
- ▶ como uma chamada recursiva ao QUICKSORT sobre um vetor de tamanho 0 nada faz, temos que $T(0) = \Theta(1)$

Separação de pior caso

- ▶ então a recorrência do tempo de execução de pior caso é

$$\begin{aligned} T(n) &= T(n-1) + T(0) + \Theta(n), \\ &= T(n-1) + \Theta(n). \end{aligned}$$

- ▶ podemos usar o método da substituição para mostrar que $T(n) = T(n-1) + \Theta(n)$ é tal que $T(n) = \Theta(n^2)$
- ▶ isso significa que se a separação é maximamente desbalanceada em todo nível da recursão, o tempo de execução é $\Theta(n^2)$
- ▶ portanto o tempo de execução de pior caso do QUICKSORT não é melhor que do INSERTIONSORT
- ▶ além disso, o tempo de execução $\Theta(n^2)$ ocorre quando o vetor de entrada está ordenado, uma situação em que o INSERTIONSORT tem tempo de execução $O(n)$

Separação de melhor caso

- ▶ no outro extremo, a separação produz dois subproblemas, cada um de tamanho não maior que $n/2$, já que um é de tamanho $\lfloor n/2 \rfloor$ e o outro é $\lceil n/2 \rceil - 1$
- ▶ neste caso o QUICKSORT é muito mais rápido
- ▶ a recorrência para o tempo de execução é então

$$T(n) = 2T(n/2) + \Theta(n)$$

- ▶ podemos usar o caso 2 do teorema mestre e obter a solução $T(n) = \Theta(n \lg n)$
- ▶ se os lados da partição são balanceados igualmente em cada nível da recursão, temos um algoritmo assintoticamente mais rápido

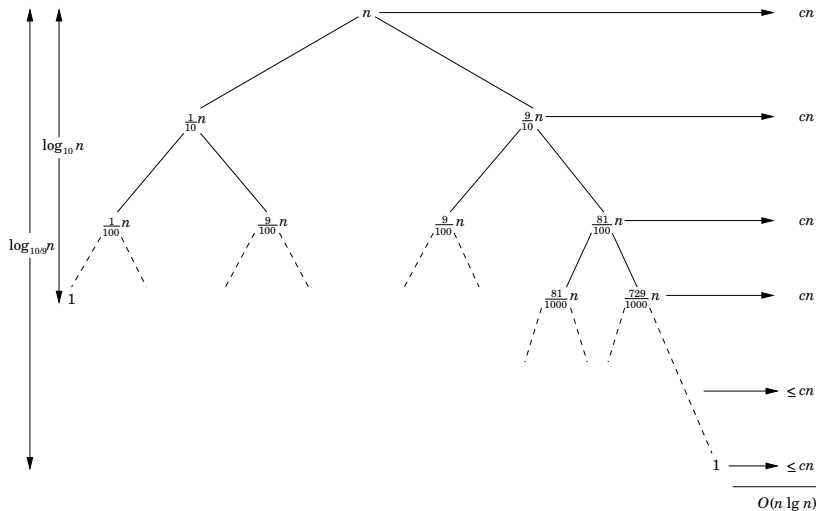
Separação balanceada

- ▶ o tempo de execução do caso médio do QUICKSORT é muito mais próximo do melhor caso do que do pior caso
- ▶ é necessário compreender como o balanceamento da separação é refletido na recorrência que descreve o tempo de execução
- ▶ suponha, por exemplo, que o algoritmo de separação sempre produz uma quebra de proporção 9-para-1, que parece bastante desbalanceada
- ▶ obtemos então a recorrência

$$T(n) = T(9n/10) + T(n/10) + cn,$$

para o tempo de execução do QUICKSORT

Árvore de recursão do algoritmo QUICKSORT



Separação balanceada

- ▶ assim, com uma quebra proporcional a 9-para-1 em todo nível da recursão, o QUICKSORT tem tempo de execução $O(n \lg n)$
- ▶ este tempo é assintoticamente o mesmo se a quebra fosse sempre no meio do vetor
- ▶ mesmo com uma quebra proporcional a 99-para-1, ainda assim o tempo de execução é $O(n \lg n)$
- ▶ qualquer quebra de proporcionalidade **constante** fornece uma árvore de recursão de profundidade $\Theta(\lg n)$, onde o custo de cada nível é $O(n)$

Intuição para o caso médio

- ▶ considere que todas as permutações dos números de entrada são igualmente prováveis
- ▶ quando executamos o QUICKSORT sobre um vetor aleatório de entrada, é altamente improvável que a separação ocorra da mesma forma em todos os níveis
- ▶ esperamos que algumas das separações sejam razoavelmente balanceadas e que algumas sejam bastante desbalanceadas
- ▶ no caso médio, o algoritmo PARTITION produz um misto de quebras “boas” e “ruins”
- ▶ na árvore de recursão para uma execução de caso médio do algoritmo PARTITION, as quebras boas e ruins são distribuídas aleatoriamente pela árvore

Ideia geral

- ▶ quando mencionamos o caso médio, supomos que todas as permutações de números de entrada são igualmente prováveis
- ▶ não podemos esperar que esta suposição sempre se concretize na prática
- ▶ em alguns casos, podemos adicionar aleatorização em um algoritmo para obter um bom desempenho esperado sobre todas as entradas
- ▶ muitas pessoas consideram a versão aleatorizada do QUICKSORT o algoritmo de ordenação a ser escolhido para entradas suficientemente grandes

Ideia geral

- ▶ usamos **amostragem aleatória** no QUICKSORT
- ▶ ao invés de sempre usar $A[r]$ como pivô, selecionamos um elemento escolhido aleatoriamente do vetor $A[p..r]$
- ▶ inicialmente, trocamos o elemento $A[r]$ com um elemento de $A[p..r]$ escolhido ao acaso
- ▶ através da amostragem aleatória no intervalo $p, \dots, r$, garantimos que o pivô $x = A[r]$ é igualmente provável a qualquer dos $r - p + 1$ elementos do vetor
- ▶ devido à escolha aleatória do pivô, esperamos que a quebra do vetor de entrada seja razoavelmente bem balanceada na média

Algoritmo aleatorizado

RANDOMIZED-PARTITION(A, p, r)

1. $i \leftarrow \text{RANDOM}(p, r)$
2. troque $A[i]$ com $A[r]$
3. **devolva** PARTITION(A, p, r)

RANDOMIZED-QUICKSORT(A, p, r)

1. **se** $p < r$
2. $q \leftarrow \text{RANDOMIZED-PARTITION}(A, p, r)$
3. RANDOMIZED-QUICKSORT($A, p, q - 1$)
4. RANDOMIZED-QUICKSORT($A, q + 1, r$)

Análise de tempo do algoritmo QUICKSORT

- ▶ vimos a intuição do comportamento de pior caso do QUICKSORT e uma justificativa do porquê o algoritmo é rápido na prática
- ▶ vamos analisar o QUICKSORT agora mais rigorosamente
- ▶ a análise de pior caso a seguir vale para o QUICKSORT e para o RANDOMIZED-QUICKSORT
- ▶ em seguida, apresentamos a análise do tempo de execução esperado do RANDOMIZED-QUICKSORT

Análise de pior caso

- ▶ podemos usar o método da substituição para mostrar que o tempo de execução do QUICKSORT é $O(n^2)$
- ▶ seja $T(n)$ o tempo de execução de pior caso do algoritmo QUICKSORT para uma entrada de tamanho n
- ▶ temos então seguinte recorrência:

$$T(n) = \max_{0 \leq q \leq n-1} (T(q) + T(n-q-1)) + \Theta(n).$$

que também pode ser descrita como:

$$T(n) = \max_{0 \leq q \leq n-1} (T(q) + T(n-q-1)) + cn,$$

para alguma constante $c > 0$

Análise de pior caso

- ▶ “chutamos” que $T(n) \leq dn^2$ para alguma constante positiva d
- ▶ substituindo na recorrência, obtemos:

$$\begin{aligned}
 T(n) &= \max_{0 \leq q \leq n-1} (T(q) + T(n-q-1)) + cn \\
 &\leq \max_{0 \leq q \leq n-1} (dq^2 + d(n-q-1)^2) + cn \\
 &\leq d(n-1)^2 + cn \\
 &\leq dn^2 - d(2n-1) + cn \\
 &\leq dn^2
 \end{aligned}$$

se $d \geq c/2$

Análise de pior caso

- ▶ assim, $T(n) = O(n^2)$
- ▶ como vimos antes, há um caso específico em que o QUICKSORT tem tempo $\Omega(n^2)$: quando a separação é desbalanceada
- ▶ portanto, o tempo de execução de pior caso do QUICKSORT é $\Theta(n^2)$

Tempo de execução esperado

- ▶ vamos mostrar que o tempo de execução esperado do algoritmo RANDOMIZED-QUICKSORT é $O(n \lg n)$
- ▶ este limitante superior para o tempo de execução esperado combinado com o tempo de execução de melhor caso $\Theta(n \lg n)$ fornece um tempo de execução esperado $\Theta(n \lg n)$ para o algoritmo
- ▶ suponha que todos os valores dos elementos a serem ordenados sejam distintos
- ▶ como os algoritmos QUICKSORT e o RANDOMIZED-QUICKSORT diferem apenas em como selecionam o pivô, vamos analisar os algoritmos QUICKSORT e PARTITION supondo que os pivôs são selecionados aleatoriamente nos subvetores

Tempo de execução esperado

- ▶ o tempo de execução do QUICKSORT é dominado pelo tempo gasto pelo algoritmo PARTITION
- ▶ cada vez que PARTITION é chamado, ele seleciona um pivô e esse elemento nunca mais é incluído em qualquer chamada recursiva futura do QUICKSORT e do PARTITION
- ▶ assim, podem haver no máximo n chamadas ao algoritmo PARTITION durante toda execução do algoritmo QUICKSORT
- ▶ uma chamada do algoritmo PARTITION gasta $O(1)$ mais uma quantidade de tempo proporcional ao número de iterações da estrutura de repetição das linhas 3–6
- ▶ cada iteração dessa estrutura executa uma comparação na linha 4 entre o pivô e um outro elemento do vetor A

Tempo de execução esperado

- ▶ portanto, se contamos o número total de vezes que a linha 4 é executada, podemos calcular o tempo total gasto na estrutura de repetição durante toda a execução do QUICKSORT

Lema

Seja X o número de comparações executadas na linha 4 do algoritmo PARTITION pela execução completa do QUICKSORT sobre um vetor de n elementos. Então, o tempo de execução do QUICKSORT é $O(n + X)$.

Tempo de execução esperado

- ▶ nosso objetivo é computar o número total de comparações X realizadas em todas as chamadas do algoritmo `PARTITION`
- ▶ não tentamos analisar quantas comparações são feitas em *cada* chamada do `PARTITION`
- ▶ ao invés disso, derivamos um limitante geral sobre o número total de comparações
- ▶ devemos, então, entender quando o algoritmo compara dois elementos do vetor e quando não compara
- ▶ vamos renomear os elementos do vetor A como $z_1, z_2, \dots, z_n$, com z_i sendo i -ésimo menor elemento
- ▶ defina $Z_{ij} = \{z_i, z_{i+1}, \dots, z_j\}$ o conjunto de elementos entre z_i e z_j , inclusive

Tempo de execução esperado

- ▶ quando o algoritmo compara z_i com z_j ?
- ▶ observe que cada par de elementos é comparado no máximo uma vez, já que os elementos são comparados apenas com o pivô e, após o término de uma chamada particular do `PARTITION`, o pivô usado naquela chamada nunca mais é comparado com qualquer outro elemento
- ▶ defina a variável aleatória indicadora

$$X_{ij} = I\{z_i \text{ é comparado com } z_j\},$$

considerando que a comparação ocorre a qualquer momento durante a execução do algoritmo, não apenas durante uma iteração ou uma chamada do `PARTITION`

Tempo de execução esperado

- ▶ como cada par é comparado no máximo uma vez, podemos caracterizar o número total de comparações realizadas pelo algoritmo como:

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}.$$

Tempo de execução esperado

- tomando a esperança de ambos os lados e usando a linearidade da esperança temos

$$\begin{aligned}
 E[X] &= E \left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij} \right] \\
 &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}] \\
 &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \Pr \{z_i \text{ é comparado a } z_j\},
 \end{aligned}$$

onde a última igualdade vale porque $E[X_B] = \Pr(B)$ para todo evento B e uma variável aleatória indicadora X_B

Tempo de execução esperado

- ▶ resta computar $\Pr\{z_i \text{ é comparado a } z_j\}$
- ▶ suponha que o algoritmo RANDOMIZED-PARTITION escolhe cada pivô aleatoriamente e independentemente
- ▶ quando dois elementos **não** são comparados?
- ▶ considere uma permutação dos números de 1 a 10 como entrada para o algoritmo QUICKSORT e suponha que o primeiro pivô seja 7
- ▶ então, a primeira chamada ao algoritmo PARTITION separa os números em dois conjuntos: $\{1, 2, 3, 4, 5, 6\}$ e $\{8, 9, 10\}$
- ▶ observe que o pivô 7 é comparado com todos os outros números, mas nenhum número do primeiro conjunto é ou será comparado a qualquer número do segundo conjunto

Tempo de execução esperado

- ▶ em geral, como consideramos que os valores dos elementos são distintos, uma vez que o pivô x é escolhido com $z_i < x < z_j$, sabemos que z_i e z_j não podem ser comparados em qualquer momento subsequente
- ▶ por outro lado, se z_i é escolhido como pivô antes de qualquer outro elemento em Z_{ij} , então z_i será comparado com cada elemento de Z_{ij} , exceto ele mesmo
- ▶ similarmente, se z_j é escolhido como pivô antes de qualquer outro elemento em Z_{ij} , então z_j será comparado com cada elemento de Z_{ij} , exceto ele mesmo
- ▶ assim, z_i e z_j são comparados se e somente se o primeiro elemento a ser escolhido como pivô em Z_{ij} é ou z_i ou z_j

Tempo de execução esperado

- ▶ antes da escolha de um pivô de Z_{ij} , o conjunto todo Z_{ij} está na mesma partição
- ▶ assim, qualquer elemento de Z_{ij} é igualmente provável de ser o escolhido como o primeiro pivô
- ▶ como o conjunto Z_{ij} tem $j - i + 1$ elementos, e como os pivôs são escolhidos aleatória e independentemente, a probabilidade que qualquer elemento seja o primeiro pivô escolhido é $1/(j - i + 1)$

Tempo de execução esperado

► assim, temos

$$\begin{aligned}
 \Pr\{z_i \text{ é comparado a } z_j\} &= \Pr\{z_i \text{ ou } z_j \text{ é escolhido primeiro pivô em } Z_{ij}\} \\
 &= \Pr\{z_i \text{ é escolhido primeiro pivô em } Z_{ij}\} \\
 &\quad + \Pr\{z_j \text{ é escolhido primeiro pivô em } Z_{ij}\} \\
 &= \frac{1}{j-i+1} + \frac{1}{j-i+1} \\
 &= \frac{2}{j-i+1}.
 \end{aligned}$$

Tempo de execução esperado

- ▶ dessa forma, voltando à esperança temos

$$\begin{aligned}
 E[X] &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \Pr\{z_i \text{ é comparado a } z_j\} \\
 &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1}.
 \end{aligned}$$

Tempo de execução esperado

- podemos avaliar essa soma fazendo uma mudança de variáveis ($k = j - i$) e limitando superiormente a série harmônica que surge na equação:

$$\begin{aligned}
 E[X] &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} \\
 &< \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k} \\
 &= \sum_{i=1}^{n-1} O(\lg n) \\
 &= O(n \lg n).
 \end{aligned}$$

Tempo de execução esperado

- ▶ assim, concluímos que, usando o algoritmo RANDOMIZED-PARTITION, o tempo de execução esperado do QUICKSORT é $O(n \lg n)$ quando os valores dos elementos são diferentes

Exercícios

- 7.1-1 Ilustre a operação do algoritmo `PARTITION` sobre o vetor $A = \langle 13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11 \rangle$.
- 7.1-2 Qual valor q o algoritmo `PARTITION` devolve quando todos os elementos no vetor $A[p..r]$ são iguais? Modifique o algoritmo tal que $q = \lfloor (p+r)/2 \rfloor$ quando todos os elementos do vetor $A[p..r]$ são iguais.
- 7.1-4 Como você modificaria o `QUICKSORT` para reorganizar o vetor de entrada em ordem decrescente?

Exercícios

- 7.2-1 Use o método da substituição para mostrar que a recorrência $T(n) = T(n-1) + \Theta(n)$ tem solução $T(n) = \Theta(n^2)$.
- 7.2-2 Qual o tempo de execução do QUICKSORT quando todos os elementos do vetor A têm o mesmo valor?
- 7.2-3 Mostre que o tempo de execução do QUICKSORT é $\Theta(n^2)$ quando o vetor A contém elementos distintos e estão arranjados em ordem decrescente.

Exercícios

- 7.3-1 Por que analisamos o tempo de execução esperado de um algoritmo aleatorizado e não seu tempo de execução de pior caso?
- 7.3-2 Quando o algoritmo `RANDOMIZED-QUICKSORT` é executado, quantas chamadas ao gerador de números aleatórios `RANDOM` são feitas no pior caso? E no melhor caso? Dê sua resposta em termos da notação Θ .

Exercícios

7.4-1 Mostre que na recorrência

$$T(n) = \max_{0 \leq q \leq n-1} (T(q) + T(n-q-1)) + \Theta(n),$$

temos que

$$T(n) = \Omega(n^2).$$

7.4-2 Mostre que o tempo de execução de melhor caso do algoritmo QUICKSORT é $\Omega(n \lg n)$.

7.4-3 Mostre que a expressão $q^2 + (n-q-1)^2$ obtém um máximo sobre $q = 0, 1, \dots, n-1$ quando $q = 0$ ou $q = n-1$.

7.4-4 Mostre que o tempo de execução esperado do algoritmo RANDOMIZED-QUICKSORT é $\Omega(n \lg n)$.

Problemas

6-1 Correção da separação de Hoare

A versão do algoritmo `PARTITION` apresentada na aula não é o algoritmo de separação originalmente proposto para o algoritmo `QUICKSORT`. O algoritmo original, devido a Charles A. R. Hoare, é apresentado a seguir.

- (a) Demonstre a operação do algoritmo `HOARE-PARTITION` sobre o vetor $A = \langle 13, 19, 9, 5, 12, 8, 7, 4, 11, 2, 5, 21 \rangle$.

Problemas

```
HOARE-PARTITION( $A, p, r$ )
01.   $x \leftarrow A[p]$ 
02.   $i \leftarrow p - 1$ 
03.   $j \leftarrow r + 1$ 
04.  enquanto VERDADEIRO
05.    faça
06.       $j \leftarrow j - 1$ 
07.      enquanto  $A[j] > x$ 
08.        faça
09.           $i \leftarrow i + 1$ 
10.      enquanto  $A[i] < x$ 
11.        se  $i < j$ 
12.          troque  $A[i]$  com  $A[j]$ 
13.      senão
14.        devolva  $j$ 
```


Problemas

As três próximas questões exigem que você forneça um argumento cuidadoso que permite mostrar que o algoritmo `HOARE-PARTITION` está correto. Considere que o vetor $A[p..r]$ contém pelo menos dois elementos. Mostre o que segue:

- (b) Os índices i e j são tais que nós nunca acessamos um elemento de A fora do vetor $A[p..r]$
- (c) Quando o algoritmo `HOARE-PARTITION` termina, ele devolve um valor j tal que $p \leq j < r$.
- (d) Cada elemento de $A[p..j]$ é menor ou igual a cada elemento de $A[j+1..r]$ quando o algoritmo `HOARE-PARTITION` termina.

Problemas

O algoritmo `PARTITION` separa o pivô, originalmente em $A[r]$, das duas partições que ele produz. Por outro lado, o algoritmo `HOARE-PARTITION` sempre coloca o pivô, originalmente em $A[p]$, em uma das duas partições $A[p..j]$ e $A[j+1..r]$. Como $p \leq j < r$, esta quebra é sempre não trivial.

- (e) Reescreva o algoritmo `QUICKSORT` usando o algoritmo `HOARE-PARTITION`.